RW BLE iOS SDK User Guide

1. Introduction

This document explains the functional APIs and usage scenarios provided by the SDK.

This document applies only to RW company Bluetooth devices.

1.1 Supported Platforms and Languages

- iOS 12 and above, Objective-C language.

1.2 Terminology

- App: Refers to the application running on a mobile phone or tablet.
- Device: Refers to wearable hardware devices, such as watches and rings.
- Upload: Refers to data sent from the device to the App.
- Download: Refers to data sent from the App to the device.

1.3 Notes

1. This document uses Objective-C for all examples. If you use Swift, you must import the corresponding Objective-C header files in the project's Bridging Header.
2. The SDK does not provide a simulator version, because BLE cannot be debugged on the simulator, and some third-party libraries used by the SDK do not support the simulator environment.

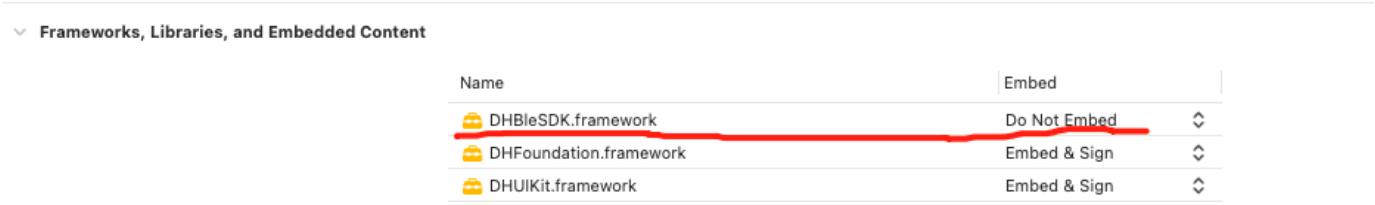
2. Quick Start

Step 1: Install the latest version of Xcode

To develop with the RW BLE iOS SDK, Xcode must be installed.

Step 2: Manually add dependencies

Manually add `DHBleSDK.framework` to your project.



Step 3: Configure info.plist

```
//Add Bluetooth usage descriptions to info.plist.
NSBluetoothAlwaysUsageDescription
NSBluetoothPeripheralUsageDescription
```

Step 4: Initialize the SDK

```
//Initialize the DHBleSDK in AppDelegate.
- (void)initBleSDK{
    [DHBleCentralManager setLogStatus:YES];
    [DHBleCentralManager initWithServiceUuids:@[]];
}
```

> **Caution**
>
> When logging is enabled `[DHBleCentralManager setLogStatus:YES]` , log files will be stored in the `Document/DeviceLog` directory.

3. API Reference)

3.1 Device Scanning, Connection, Binding, and Reconnection

3.1.1 Scan for Devices

> Description: Call `startScan` to scan for BLE devices and implement the `DHBleConnectDelegate`.
>
> If the returned `DHPeripheralModel` has an empty `macAddr`, it indicates that the device is already paired in system settings.

```
// 1. 开始搜索
[DHBleCentralManager startScan];

//2. 设备委托
[DHBleCentralManager shareInstance].connectDelegate = self;

//3. DHBleConnectDelegate接口会回调搜索到的蓝牙设备
- (void)centralManagerDidDiscoverPeripheral:(NSArray <DHPeripheralModel *>*)peripherals
```

3.1.2 Stop Scanning

> Description: Stop scanning for BLE devices.

```
[DHBleCentralManager stopScan];
```

3.1.3 Connect Device

> Description: Connect to a specified device and set the connection delegate.

```
// 1. Initialize and register the callback.
[DHBleCentralManager shareInstance].connectDelegate = self;

// 2. Connect device
DHPeripheralModel *deviceModel = self.deviceArray[indexPath.row];
[DHBleCentralManager connectDeviceWithModel:deviceModel];

// 3. Implement and receive callbacks for Bluetooth connection status.
@protocol DHBleConnectDelegate <NSObject>
```

`DHBleConnectDelegate` Interface Description:

method	illustrate
centralManagerDidDiscoverPeripheral	After calling `startScan`, the system will return a callback when a device is found.
centralManagerDidConnectPeripheral	After calling `connectDeviceWithModel`, the function will return upon successful connection.
centralManagerDidFunctionMenu	The function will return after successfully obtaining the device configuration table; business operations should be performed after this point.
centralManagerDidDisconnectPeripheral	A callback function will be triggered when the Bluetooth connection is disconnected.
centralManagerDidFailedPeripheral	Bluetooth failure will trigger a callback.
centralManagerDidUpdateState	The Bluetooth switch state change will trigger a callback.

> **Tip**
>
> After connecting, business operations should only be performed after the `centralManagerDidFunctionMenu` method has been called.

(1) Listening to `BluetoothNotificationConnectStateChange` can also provide information about changes in the connection state.

```
[[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(connectStateChange:)
name:BluetoothNotificationConnectStateChange object:nil];

- (void)connectStateChange:(NSNotification *)ntf
{
  NSLog(@"NewHomeController connectStateChange");
  if ([DHBluetoothManager shareInstance].isConnected){
    self.infoDeviceStateLb.text = @"Connected";
  }
  else{
    self.infoDeviceStateLb.text = @"Disconnected";
  }
}
```

(2) `[DHBleCentralManager isConnected]` can be used to check if the connection was successful.

3.1.4 Disconnect the device.

> Interface description: Disconnect the currently connected device;

```
[DHBleCentralManager disconnectDevice];
```

3.1.5 Binding and automatic reconnection, unbinding

3.1.5.1 DHBleCentralManager Save to the current device locally.

> After setting this, the current device's UDID will be saved, and it will automatically reconnect.

Method Description:

`+(void)setBindedStatus:(BOOL)isBinded`

Example of usage:

```
//Save locally, and reopening the file will reconnect.
[DHBleCentralManager setBindedStatus:YES];
```

3.1.5.2 DHBleCentralManager Delete locally saved data

> After this setting is applied, the UDID of the current device will be deleted, similar to unlinking the device locally.

Method Description:

`+(void)setBindedStatus:(BOOL)isBinded`

Example of usage:

```
[DHBleCentralManager setBindedStatus:NO];
[DHBleCentralManager disconnectDevice];
```

3.1.6 Equipment Configuration Table

> **Important**
>
> Due to the variety of device models and their differing supported features, a feature table has been introduced to allow users to check the supported functions of each device. Please refer to the DeviceFuncV2Model class for details. The feature table content can be saved according to your specific business needs.

`-(void)centralManagerDidFunctionMenu:(DeviceFuncV2Model *)deviceFuncModel`

DeviceFuncV2Model class attribute definitions:

attribute	illustrate
isPushMsgEnableSwitch	Enable or disable message control switch
pushMsgSwitchValue	Supported message types, low 32 bits (bit0-bit31)
pushMsgSwitchValue2	Supported message types, high 32 bits (bit32-bit63); defaults to 0 on old devices
activityDataInterval	Today's step-detail interval in minutes; an unconfigured value is normalized to 60
isAlarm	Does it support an alarm clock?
isBackLight	Does it support screen sleep time settings?
isSupportWorkout3	Does it support multiple sports modes?
isSupportMuslimCountSwitch	Does it support enabling/disabling Muslim prayers?
isSupportHrSp02Alert	Does it support HR and SpO2 alarm notification functions?
isSupportMotoVibrationLevel	Does it support motor vibration alerts?
isSupportAlarmVibrationDuration	Does it support alarm vibration duration setting?
isSupportVibrationInterval	Does it support vibration interval setting?
isDataTypeActivity	Does it support step counting?

isDataTypeHeart	Does it support heart rate monitoring?
isDataTypeBloodPressure	Does it support blood pressure measurement?
isDataTypeSleep	Does it support sleep mode?
isDataTypeSPO2	Does it support blood oxygen monitoring?
isDataTypeHRV	Does it support heart rate variability?
isDataTypeStress	Does it support pressure sensitivity?
isDataTypeBloodSugar	Does it support blood glucose monitoring?
isDataTypeMuslimCount	Do you support giving compliments/praise?
isSupportMuslimTimeDisplayMode	Does it support Muslim time display mode?
isSupportSensorRawPPG	Does it support PPG Green raw data?
isSupportPPGMonitoring	Does it support PPG timed monitoring?
isSupportTemperatureMonitoring	Does it support temperature timed monitoring?
isSupportCountReminder	Does it support count reminder interval setting?
isSupportSensorRawACC	Does it support ACC raw data?
isSupportSensorRawPPGRed	Does it support PPG Red raw data?
isSupportSensorRawIR	Does it support IR (infrared) raw data?
isSupportSensorRawSleep	Does it support sleep real-time data?
isSupportFallDetect	Does it support fall detection alert?
isSupportRecording	Does it support recording function?

3.1.8 Use an External CBCentralManager for Scanning and Let the SDK Connect

> Description: If the customer App integrates multiple BLE SDKs and uses its own unified BLE scanning entry, pass the scanning `CBCentralManager` to this SDK. The customer App is only responsible for scanning. Connection, service discovery, characteristic discovery, data transfer, and disconnection are still handled by the SDK.

> **Important**
>
> A `CBPeripheral` is bound to the `CBCentralManager` that discovered it. The `peripheral` passed to the SDK must come from the currently set `externalCentralManager`. The SDK does not use private APIs to verify the source. If the Central and Peripheral do not match, the connection may fail or time out.

Method Description:

```
// Initialize the SDK and specify an external CBCentralManager.
// Pass nil to use the SDK-managed mode.
+ (void)initWithServiceUuids:(NSArray <NSString *>*)uuids
      externalCentralManager:(nullable CBCentralManager *)centralManager;

// Set an external CBCentralManager at runtime.
// It can only be set when the SDK is not connecting or connected.
+ (void)setExternalCentralManager:(nullable CBCentralManager *)centralManager;
```

```objc
// Check whether the SDK is using an external CBCentralManager.
+ (BOOL)isUsingExternalCentralManager;

// Connect to a device discovered by the customer's unified scanner.
+ (void)connectPeripheral:(CBPeripheral *)peripheral
        advertisementData:(nullable NSDictionary *)advertisementData
                     RSSI:(nullable NSNumber *)RSSI;
```

Parameter Description:

parameter	type	description
centralManager	CBCentralManager	The same Central used by the customer App for unified scanning. The SDK will use it to connect.
peripheral	CBPeripheral	The device object discovered by `centralManager`. The connection must use this object.
advertisementData	NSDictionary	Advertisement data used to complete `DHPeripheralModel` fields such as `macAddr`, `deviceModel`, and device name. It is not required for the connection itself and can be nil.
RSSI	NSNumber	Signal strength used to complete `DHPeripheralModel.rssi`. It is not required for the connection itself and can be nil.

Example:

```objc
// 1. The customer App scans devices with its own CBCentralManager.
// centralManager, peripheral, advertisementData, and RSSI come from the customer's scan callback.

// 2. Pass the same CBCentralManager to the SDK.
[DHBleCentralManager initWithServiceUuids:@[]
                   externalCentralManager:centralManager];

// Or set the external Central at runtime if the SDK has already been initialized.
[DHBleCentralManager setExternalCentralManager:centralManager];

// 3. Set the SDK connection delegate.
[DHBleCentralManager shareInstance].connectDelegate = self;

// 4. Let the SDK start the connection.
[DHBleCentralManager connectPeripheral:peripheral
                     advertisementData:advertisementData
                                  RSSI:RSSI];
```

Notes:

- The old integration method `[DHBleCentralManager initWithServiceUuids:@[]]` remains unchanged, and the SDK will create its own `CBCentralManager`.
- In external Central mode, the SDK connects by using the same `CBCentralManager` passed by the customer App.
- After the SDK starts connecting, it takes over `centralManager.delegate`, and subsequent connection callbacks are handled by the SDK.
- The customer App should stop its own scanning flow before handing the device to the SDK for connection.
- Do not switch `externalCentralManager` while connecting, connected, discovering services, or synchronizing data.

- For automatic reconnection in external Central mode, the customer App must keep the passed `centralManager` instance alive.

3.2 Device function operation

Error code definitions for API calls SendStateCode:

SendStateCode	value	illustrate
SendState_OK	0	OK
SendState_BLENoOpen	1	Failure, Bluetooth is not enabled.
SendState_BLEDisconnect	2	Failure, Bluetooth not connected.
SendState_TimeOut	3	Failure, Timeout
SendState_Failed	4	fail
SendState_DuplicateSend	5	Repeat command
SendState_NotSupport	6	Device not supported

3.2.1 Basic function command interface

3.2.1.0 Get SDK Version

> Get the SDK version number.

Method Description:

`[DHBleCommand getSDKVersion]`

Example of usage:

```
NSLog(@"%@", [DHBleCommand getSDKVersion]);
```

3.2.1.1 Get Bluetooth MAC address

> Because the iOS system cannot obtain the MAC address from the broadcast packet after pairing, this method is provided to retrieve it.

Method Description:

`+ (void)ringGetMacAddress:(void(^)(int code, id data))block`

Return parameter description:

parameter	type	illustrate	
model	DHDeviceInfoModel	class	macAddr: Mac地址

Example of usage:

```
[DHBleCommand ringGetMacAddress:^(int code, id  _Nonnull data) {
  DHDeviceInfoModel *tDeviceInfoData = data;
  NSLog(@"mac: %@", tDeviceInfoData.macAddr);
}];
```

3.2.1.2 Set user information

> User information settings are related to step count, calories burned, and distance. During device initialization, the default settings are: gender 1, age 18, height 170cm, and weight 65 kg.

Method Description:

```
+ (void)setUserInfo:(DHUserInfoSetModel *)model block:(void(^)(int code, id data))block
```

Parameter Description:

parameter	type		illustrate
model	DHUserInfoSetModel	class	gender: Gender (0. Female, 1. Male) Height: height in cm, floating-point number Weight: weight in kg, floating-point number Step Goal: step count target value, currently no functionality Age: age

Example of usage:

```
DHUserInfoSetModel *userInfoModel = [[DHUserInfoSetModel alloc] init];
userInfoModel.gender = 1;
userInfoModel.height = 170;
userInfoModel.weight = 600;
userInfoModel.stepGoal = 8000;
userInfoModel.age = 20;
[DHBleCommand setUserInfo:userInfoModel block:^(int code, id  _Nonnull data) {
  if (code == 0){
    NSLog(@"set ok");
  }
}];
```

3.2.1.3 Get firmware information

> Interface description: Retrieves the firmware model, firmware version number, and UI version number;

```
[DHBleCommand getFirmwareVersion:^(int code, id  _Nonnull data) {
            if (code == 0){
                NSLog(@"getFirmwareVersion OK");
                DHFirmwareVersionModel *model = data;
                NSLog(@"model %@ version %@ UI version %@", model.deviceModel, model.firmwareVersion,
model.uiVersion);
            }
        }];
```

3.2.1.4 Get battery level

> Interface description: App retrieves device battery level.

```
[DHBleCommand getBattery:^(int code, id  _Nonnull data) {
            if (code == 0){
                DHBatteryInfoModel *model = data;
                NSLog(@"getBattery OK battery %zd", model.battery);
            }
        }];
```

3.2.1.5 Get and set video control switches.

> Configure whether to enable ring gesture control for browsing videos; This function requires pairing with a Bluetooth HID device.

```
//Set video control switch
DHVideoHidSetModel *tModeSetModel = [[DHVideoHidSetModel alloc] init];
tModeSetModel.isOpen = YES;
[DHBleCommand setVideoHid:tModeSetModel block:^(int code, id  _Nonnull data) {
  if (code == 0){
    NSLog(@"setVideoHid OK");
  }
}];

// Get video control switch
[DHBleCommand getVideoHid:^(int code, id  _Nonnull data) {
  if (code == 0){
    DHVideoHidSetModel *model = data;
    NSLog(@"getVideoHid OK isOpen %d", model.isOpen);
  }
}];
```

3.2.1.6 Get and set LED screen brightness.

Method Description:

`+(void)getRingLEDLight:(void(^)(int code, id data))block;`

`+(void)setRingLEDLight:(DHLedLightSetModel *)model block:(void(^)(int code, id data))block;`

Parameter Description:

parameter	type		illustrate
model	DHLedLightSetModel	class	isOpen: false means off, true means (Levels 1-3) Light Level: 1 (dim light), 2 (soft light), 3 (bright light)

Example of usage:

```
// Get the LED screen brightness level.
[DHBleCommand getRingLEDLight:^(int code, id  _Nonnull data) {
  if (code == 0){
    DHLedLightSetModel *model = data;
    NSLog(@"getRingLEDLight OK 开关 %d", model.isOpen);
  }
}];
```

```
// Set the LED screen brightness.
DHLedLightSetModel *tModeSetModel = [[DHLedLightSetModel alloc] init];
tModeSetModel.isOpen = YES; //
tModeSetModel.lightLevel = 3; //1 (dim light), 2 (soft light), 3 (bright light)
[DHBleCommand setRingLEDLight:tModeSetModel block:^(int code, id  _Nonnull data) {
  if (code == 0){
    NSLog(@"setRingLEDLight OK");
  }
}];
```

3.2.1.7 Get and set the wearing position.

```
// 获取佩戴位置
[DHBleCommand getRingWearHand:^(int code, id  _Nonnull data) {
  if (code == 0){
    NSInteger tWearHand = [data intValue]; //0左手 1右手
    NSLog(@"getRingWearHand OK 佩戴位置 %zd", tWearHand);
  }
}];

//设置佩戴位置
uint8_t tModeSetModel = 0; 0: Left hand  1: Right hand
[DHBleCommand setRingWearHand:tModeSetModel block:^(int code, id  _Nonnull data) {
  if (code == 0){
    NSLog(@"setRingWearHand OK");
  }
}];
```

3.2.1.8 Starting and stopping photo taking

> After activating the camera function, the device can use gesture control to take photos with the customized camera app.
>
> Configuration table attribute: `isTakePhoto`
>
> The `BluetoothNotificationCameraTakePicture` device sends a notification to take a picture, and then takes the picture.

```
// When the app enters the camera interface, a value of 1 controls the device to enter the
corresponding interface, and a value of 0 controls the device to exit.
[DHBleCommand controlCamera:1 block:^(int code, id  _Nonnull data) {

}];

// The monitoring device issued a command to take a picture.
[[NSNotificationCenter defaultCenter] addObserver:self
selector:@selector(cameraTakePictureNotification) name:BluetoothNotificationCameraTakePicture
object:nil];

- (void)cameraTakePictureNotification {
  // Take a photo
  NSLog(@"cameraTakePictureNotification Take a photo");
}
```

3.2.1.9 Find devices

> After initiating the search function, the device's light or screen will turn on.

Method Description:

`+(void)controlFindDeviceBegin:(void(^)(int code, id data))block;`

Example of usage:

```
[DHBleCommand controlFindDeviceBegin:^(int code, id  _Nonnull data) {

}];
```

3.2.1.10 Turn off the device, restore factory settings.

Method Description:

`+(void)controlDevice:(NSInteger)type block:(void(^)(int code, id data))block;`

Parameter Description:

parameter	type		illustrate
type	NSInteger	整形	1: Turn off 2: restore factory

Example of usage:

```
// 1关机 2 恢复出厂
[DHBleCommand controlDevice:1 block:^(int code, id  _Nonnull data) {

}];
```

3.2.1.11 Alarm

Configuration table attribute: `isAlarm`

3.2.1.11.1 Get the alarms that have been set.

Method Description:

`+(void)getAlarms:(void(^)(int code, id data))block`

Example of usage:

```
//获取设备里已保存的闹钟
[DHBleCommand getAlarms:^(int code, id  _Nonnull data) {
  if (code == 0) {
    NSArray *tAlarmList = data;
    NSLog(@"getAlarms %zd", tAlarmList.count);
  }
}];
```

3.2.1.11.2 Set an alarm

> **The current protocol does not support modifying individual alarms. Any operation to switch on/off or delete a single alarm requires resending the entire alarm configuration.**

Method Description:

```
+(void)setAlarms:(NSArray <DHAlarmSetModel *>*)alarms block:(void(^)(int code, id data))block;
```

Parameter Description:

parameter	type	illustrate	illustrate
alarms	List	List	isOpen: true Open/false Off repeats: IntArray(7) Sunday to Saturday, set the corresponding element to 1 for repetition. hour: start hour minute: start minute

Example of usage:

```
//设置闹钟
DHAlarmSetModel *tAlarm1 = [[DHAlarmSetModel alloc] init];
tAlarm1.hour = 07; //时
tAlarm1.minute = 00;//分
tAlarm1.isOpen = true;//开关
tAlarm1.repeats = @[@(0),@(0),@(0),@(0),@(0),@(0),@(1)]; //重复周期,周六重复

DHAlarmSetModel *tAlarm2 = [[DHAlarmSetModel alloc] init];
tAlarm2.hour = 8;
tAlarm2.minute = 00;
tAlarm2.isOpen = true;
tAlarm2.repeats = @[]; //单次闹钟

[DHBleCommand setAlarms:@[tAlarm1, tAlarm2] block:^(int code, id  _Nonnull data) {

}];
```

3.2.1.11.3 Delete all alarms

> By passing an empty array to the `setAlarms` parameter, you can delete all alarms.

```
[DHBleCommand setAlarms:@[] block:^(int code, id  _Nonnull data) {

}];
```

3.2.1.12 Setting and retrieving the number of vibrations

> Set the number of times the device vibrates;
>
> Configuration table property: `isSupportMotoVibrationLevel`

Method Description:

```
+(void)setRingMotorLevel:(NSInteger)motorLevel motorNum:(NSInteger)motorNum block:(void(^)(int code, id data))block;
```

```
+(void)getRingMotorLevel:(void(^)(int code, id data))block;
```

Parameter Description:

parameter	type	illustrate	illustrate
motorLevel	Int	整形	Vibration intensity: 0: Off, 1: Low, 2: Medium, 3: High; *This function is not defined and can be ignored*
motorNum	Int	整形	The number of vibrations can be set (0-6 times), with a default setting of 2 times. Setting it to 0 will disable vibration.

Example of usage:

```
//设置
[DHBleCommand setRingMotorLevel:1 motorNum:2 block:^(int code, id  _Nonnull data) {
  if (code == 0){
    NSLog(@"set ok");
  }
}];

//获取
[DHBleCommand getRingMotorLevel:^(int code, id  _Nonnull data) {
  if (code == 0){
    DHVibrationLevelModel *tVibrationModel = data;
    NSLog(@"getRingMotorLevel Level %d num %d", tVibrationModel.vibrationLevel,
tVibrationModel.vibrationNumber);
  }
}];
```

3.2.1.13 Setting and retrieving screen sleep mode settings.

> Set screen sleep mode and time;
>
> Configuration table property: `isBackLightSleepMode`

Method Description:

`+(void)setDisplaySleepMode:(DHBrightTimeSetModel *)model block:(void(^)(int code, id data))block`

`+(void)getDisplaySleepMode:(void(^)(int code, id data))block`

Parameter Description:

parameter	type	illustrate	illustrate
DHBrightTimeSetModel	class		sleepOpen: Switch on (YES) or off (NO) sleepStartHour: Start time (hour) sleepStartMin: Start time (minute) sleepEndHour: End time (hour) sleepEndMin: End time (minute)

Example of usage:

```
//设置
DHBrightTimeSetModel *sleepModel = [[DHBrightTimeSetModel alloc] init];
sleepModel.sleepOpen = YES;
```

```
sleepModel.sleepStartHour = 20;
sleepModel.sleepStartMin = 00;
sleepModel.sleepEndHour = 06;
sleepModel.sleepEndMin = 00;
[DHBleCommand setDisplaySleepMode:sleepModel block:^(int code, id  _Nonnull data) {

}];

//获取
[DHBleCommand getDisplaySleepMode:^(int code, id  _Nonnull data) {
  if (code == 0){
    DHBrightTimeSetModel *tModel = data;
    NSLog(@"getDisplaySleepMode sleepOpen %d sleepStartHour %d sleepStartMin %d", tModel.sleepOpen,
tModel.sleepStartHour, tModel.sleepEndMin);
  }
}];
```

3.2.1.14 Setting and retrieving notification push settings

> `DeviceFuncV2Model.isPushMsgEnableSwitch` indicates whether the device supports message-push switch settings. `pushMsgSwitchValue` and `pushMsgSwitchValue2` are capability masks for bit0-bit31 and bit32-bit63; they are not the device's current switch states.
>
> When opening the message-push settings page, call `ringGetAncs:` to fetch the current switches from the device. On success, `data` is a `DHAncsSetModel`. Modify that model and call `ringSetAncs:block:` to write the switches back to the device.

Purpose	Property/API	Result
Check feature support	`DeviceFuncV2Model.isPushMsgEnableSwitch`	Whether message-push switches are supported
Check supported message types	`pushMsgSwitchValue` / `pushMsgSwitchValue2`	Supported message types, bit0-bit63
Fetch current switches	`ringGetAncs:`	`DHAncsSetModel`
Set current switches	`ringSetAncs:block:`	Setting result

Method Description:

`+(void)ringSetAncs:(DHAncsSetModel *)model block:(void(^)(int code, id data))block`

`+(void)ringGetAncs:(void(^)(int code, id data))block`

Parameter Description:

parameter	type	illustrate	illustrate
DHAncsSetModel	class		See the definition of the DHAncsSetModel class.

Example of usage:

```
//设置
```

```
DHAncsSetModel *tAncsModel = [[DHAncsSetModel alloc] init];
tAncsModel.isSMS = YES;
[DHBleCommand ringSetAncs:tAncsModel block:^(int code, id  _Nonnull data) {

}];

//获取
[DHBleCommand ringGetAncs:^(int code, id  _Nonnull data) {
  if (code == 0){
    DHAncsSetModel *ancsModel = data;
    NSLog(@"SMS=%d Wechat=%d", ancsModel.isSMS, ancsModel.isWechat);
  }
}];

//按下面的顺序确定当前设备支持哪些应用消息.
UInt8 tBitRow = i;
if (i == 24){ //其它
  tBitRow = 0;
}
if (i > 0 && (tMsgSwitchValue & (1 << tBitRow)) < 1){ //不支持的消息全设置为false,不管从设备读到的值.
  continue;
}

self.msgOpenFlagArr = [NSMutableArray arrayWithArray:@[
  @(self.model.isCall),
  @(self.model.isSMS),
  @(self.model.isEmail),
  @(self.model.isSkype),
  @(self.model.isFacebook),
  @(self.model.isWhatsapp),
  @(self.model.isLine),
  @(self.model.isInstagram),
  @(self.model.isKakaotalk),
  @(self.model.isGmail),
  @(self.model.isTwitter),
  @(self.model.isLinkedin),
  @(self.model.isJLSinaWeiBo),

  @(self.model.isQQ),
  @(self.model.isWechat),
  @(self.model.isJLBand),
  @(self.model.isJLTelegram),
  @(self.model.isJLBetween),
  @(self.model.isJLNavercafe),
  @(self.model.isYoutube),
  @(self.model.isJLNetflix), //(1<<21)
  @(self.model.isMax), // (1<<22)
  @(self.model.isVkim), // (1<<23)
  //        @(self.model.isMessenger),

  @(self.model.isOther)
]];
```

3.2.1.15 Get and set whether the "likes" feature is enabled.

> Set whether the "like" function is enabled;
>
> Configuration table property: `isSupportMuslimCountSwitch`

Method Description:

`+(void)setMuslimCountSwitch:(UInt8)isOpen block:(void(^)(int code, id data))block`

`+(void)getMuslimCountSwitch:(void(^)(int code, id data))block`

Parameter Description:

parameter	type	illustrate	illustrate
isOpen	Int		0: Off 1: Open

Example of usage:

```
//设置
[DHBleCommand setMuslimCountSwitch:1 block:^(int code, id  _Nonnull data) {

}];

//获取
[DHBleCommand getMuslimCountSwitch:^(int code, id  _Nonnull data) {
  if (code == 0){
    Boolean tOpen = [data boolValue];
    NSLog(@"getMuslimCountSwitch tOpen %d", tOpen);
  }
}];
```

3.2.1.16 Get and set heart rate/blood oxygen alarm configuration.

> This function allows you to set heart rate and blood oxygen level notification and alarm data; the alarm notification will be sent via `BluetoothNotificationRingHealthOverAlert`.
>
> Configuration table attribute: `isSupportHrSp02Alert`

Method Description:

`+(void)getHRAlert:(void(^)(int code, id data))block`

`+(void)setHRAlert:(DHHRAlertModel *)overModel block:(void(^)(int code, id data))block`

`+(void)getSP02Alert:(void(^)(int code, id data))block`

`+(void)setSP02Alert:(DHHRAlertModel *)overModel block:(void(^)(int code, id data))block`

Parameter Description:

Parameter	type	illustrate	illustrate
DHHRAlertModel	class		isOpen: YES (On), NO (Off); overValue: Alarm threshold, default values are heart rate exceeding 160 and blood oxygen below 94%. underValue: An alarm will be triggered if the value is below the set threshold; if the retrieved value is 0xff, it indicates that this function is not supported.

Note: If the `underValue` returned by `getHRAlert()` is 0xff, it means this feature is not supported.

Example of usage:

```
//set
DHHRAlertModel *tHRAlertModel = [[DHHRAlertModel alloc] init];
tHRAlertModel.isOpen = YES;
tHRAlertModel.overValue = 160;
tHRAlertModel.underValue = 0xff;
[DHBleCommand setHRAlert:tHRAlertModel block:^(int code, id  _Nonnull data) {

}];

//get
[DHBleCommand getHRAlert:^(int code, id  _Nonnull data) {
  if (code == 0){
    DHHRAlertModel *model = data;
    NSLog(@"getHRAlert %d %zd", model.isOpen, model.overValue);
  }
}];


// Alarm notification push
    [[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(healthOverAlert:)
name:BluetoothNotificationRingHealthOverAlert object:nil];

- (void)healthOverAlert:(NSNotification *)ntf
{
    NSDictionary *tUserInfo = ntf.userInfo;
    NSInteger tType = [tUserInfo[@"type"] integerValue];
    NSInteger tValue = [tUserInfo[@"value"] integerValue];

    if (tType == 0){ //HeartRate Alert

    }
    else if (tType == 1){ //SP02

    }
}
```

3.2.1.17 Get and set screen timeout duration.

> Configuration table property: `isBackLight`

Method Description:

`+(void)getBrightTime:(void(^)(int code, id data))block`

`+(void)setBrightTime:(DHBrightTimeSetModel *)model block:(void(^)(int code, id data))block`

Parameter Description:

DHBrightTimeSetModel Parameter	Type	Description	Value
duration	Int	Screen-on duration, in seconds (s), range 0-30s;	
durationNums	String	Supported duration values by the device; if available, separated by commas;	

3.2.1.18 Get and Set Wrist-Raise Screen-On Duration

> Configuration table property: `isSupportRaisescreen`

Method Description:

```
+(void)ringGetGesture:(void(^)(int code, id data))block;
```

```
+(void)ringSetGesture:(DHGestureSetModel *)model block:(void(^)(int code, id data))block;
```

Parameter Description:

BrightScreenBean Parameter	Type	Description	Value
isOpen	Int	true: enabled; false: disabled	
startHour	Int	Start time (hour)	
startMin	Int	Start time (minute)	
endHour	Int	End time (hour)	
endMin	Int	End time (minute)	

3.2.1.19 Set Time Format (12-Hour / 24-Hour)

> This setting only applies to devices with a display.

Method Description:

```
+(void)ringSetTimeformat:(UInt8)timeformat block:(void(^)(int code, id data))block;
```

Parameter Description:

Parameter	Type	Description	
timeformat	Int	0: 24-Hour 1: 12-Hour	

3.2.1.20 Alarm Vibration Duration Setting and Getting

> Set the alarm vibration count;
>
> Configuration table property: `isSupportAlarmVibrationDuration`

Method Description:

`+(void)setAlarmVibrationDuration:(UInt8)count block:(void(^)(int code, id data))block`

`+(void)getAlarmVibrationDuration:(void(^)(int code, id data))block`

Parameter Description:

Parameter	Type	Description	Value
count	UInt8	Integer	Vibration count (0-6), default 2, 0 means no vibration

Example of usage:

```
//Set
[DHBleCommand setAlarmVibrationDuration:2 block:^(int code, id  _Nonnull data) {
    if (code == 0){
        NSLog(@"setAlarmVibrationDuration OK");
    }
}];

//Get
[DHBleCommand getAlarmVibrationDuration:^(int code, id  _Nonnull data) {
    if (code == 0){
        NSLog(@"getAlarmVibrationDuration count %@", data);
    }
}];
```

3.2.1.21 Touch Event Notification

> Device touch event notification, actively reported by the device. Touch operations are reported regardless of screen state. The APP defines the response behavior.
>
> Received via `BluetoothNotificationTouchEvent` notification.

Notification userInfo data:

Field	Description	Value
keyType	Key type	1: Touch key (default), 2: Fall (requires fall detect enabled 3.2.1.24)
touchType	Touch type	1: Single tap, 2: Double tap, 3: Triple tap, 4: Long press, 5: Flick. When key type=2 (fall), touch type defaults to 1

Example of usage:

```
[[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(touchEventNotification:)
name:BluetoothNotificationTouchEvent object:nil];

- (void)touchEventNotification:(NSNotification *)ntf
{
    NSDictionary *tUserInfo = ntf.userInfo;
    NSInteger keyType = [tUserInfo[@"keyType"] integerValue];   // 1: Touch key
    NSInteger touchType = [tUserInfo[@"touchType"] integerValue]; // 1: Single tap, 2: Double tap, 3:
Triple tap, 4: Long press, 5: Flick
    NSLog(@"TouchEvent keyType=%zd touchType=%zd", keyType, touchType);
}
```

3.2.1.22 Vibration Interval Setting and Getting

> Set the interval time between each vibration, used to adjust vibration rhythm;
>
> Configuration table property: `isSupportVibrationInterval`

Method Description:

`+(void)setVibrationInterval:(UInt16)intervalMs block:(void(^)(int code, id data))block`

`+(void)getVibrationInterval:(void(^)(int code, id data))block`

Parameter Description:

Parameter	Type	Description	Value
intervalMs	UInt16	Integer	Interval duration (100-1000ms), default 500ms

Example of usage:

```
//Set
[DHBleCommand setVibrationInterval:500 block:^(int code, id  _Nonnull data) {
    if (code == 0){ NSLog(@"setVibrationInterval OK"); }
}];

//Get
[DHBleCommand getVibrationInterval:^(int code, id  _Nonnull data) {
    if (code == 0){ NSLog(@"getVibrationInterval %@ms", data); }
}];
```

3.2.1.23 HR Calibration (Factory Test)

> Start device heart rate calibration mode. After sending the calibration command, the device returns 2 responses:
>
> 1st response: result=0 (calibrating); 2nd response: result≠0 (calibration done).
>
> block callback data is NSDictionary: `testMode`(UInt8) + `result`(UInt32, 0=calibrating, non-0=done).

Method Description:

`+(void)startFactoryTest:(UInt8)testMode block:(void(^)(int code, id data))block`

Parameter Description:

Parameter	Type	Description	Value
testMode	UInt8	Test mode	0x15: HR Calibration

Example of usage:

```
[DHBleCommand startFactoryTest:0x15 block:^(int code, id  _Nonnull data) {
    if (code == 0 && [data isKindOfClass:[NSDictionary class]]){
        NSDictionary *info = data;
        NSInteger result = [info[@"result"] integerValue];
        if (result == 0){
            NSLog(@"HR calibrating...");
        } else {
            NSLog(@"HR calibration done, result=%zd", result);
        }
    }
}];
```

3.2.1.24 Fall Detection Setting

> Set or get the fall detection alert switch. When enabled, the device will report fall events via touch event notification (3.2.1.21).
>
> Fall events are reported through `BluetoothNotificationRingTouchEvent` notification, with keyType=2 indicating a fall event.
>
> Configuration table property: `isSupportFallDetect`

Method Description:

`+(void)setFallDetect:(UInt8)enable block:(void(^)(int code, id data))block`

`+(void)getFallDetect:(void(^)(int code, id data))block`

Parameter Description:

Parameter	Type	Description	Value
enable	UInt8	Switch	0: off, 1: on

Example of usage:

```
//Get fall detect switch
[DHBleCommand getFallDetect:^(int code, id  _Nonnull data) {
    if (code == 0){
        NSLog(@"getFallDetect OK: %@", data);
    }
}];

//Set fall detect on
[DHBleCommand setFallDetect:1 block:^(int code, id  _Nonnull data) {
    if (code == 0){
        NSLog(@"setFallDetect ON OK");
    }
}];
```

3.2.1.25 Count Reminder Interval Setting

> Set or get the count reminder interval. When enabled, after the user completes a count operation, the device starts timing and vibrates once when the interval is reached to remind the user to continue counting.
>
> Configuration table property: `isSupportCountReminder`

Method Description:

`+(void)setCountReminderInterval:(UInt8)interval block:(void(^)(int code, id data))block`

`+(void)getCountReminderInterval:(void(^)(int code, id data))block`

Parameter Description:

Parameter	Type	Description	Value
interval	UInt8	Interval minutes	0: off, 30/60/90/120: reminder interval

Example of usage:

```
//Get count reminder interval
[DHBleCommand getCountReminderInterval:^(int code, id  _Nonnull data) {
    if (code == 0){
        NSLog(@"CountReminderInterval: %@ min", data);
    }
}];

//Set count reminder interval to 60 minutes
[DHBleCommand setCountReminderInterval:60 block:^(int code, id  _Nonnull data) {
    if (code == 0){
        NSLog(@"setCountReminderInterval OK");
    }
}];
```

3.2.2 Health data synchronization (real-time single measurement and all-day monitoring)

> There are two ways to monitor health data: real-time single measurements and continuous 24-hour monitoring. Health data includes heart rate, blood oxygen, stress levels, HRV, and sleep, **but sleep is not monitored in real time**.
>
> (1) Real-time single detection: The app initiates a single detection on the device, and the results are returned immediately after the detection is complete.
>
> (2) Continuous monitoring: You can set the interval time, for example, 30 minutes or 60 minutes, and the device will perform measurements and save the data; **if the app is not synchronized, the device can store 3-6 days of data.**

3.2.2.1 Real-time monitoring - Start and stop device health data monitoring.

> Start health data monitoring (heart rate, blood oxygen, HRV, stress, blood sugar);
>
> After the test is completed, the device notifies the app via the `BluetoothNotificationHealthRingMeasureStateChange` notification;

> The real-time test values are notified to the app via the `BluetoothNotificationHealthRingMeasureValueChange` notification;

> **Caution**
>
> Only one health detection type can be active at a time. You must wait for the current detection to complete (receive the completion callback) or manually stop it before starting a new detection type. Starting multiple types simultaneously will cause detection errors.

Method Description:

`+(void)controlOpen:(NSInteger)type dataType:(NSInteger)dataType block:(void(^)(int code, id data))block`

Parameter Description:

Parameter	Type	illustrate	illustrate
dataType	NSInteger	Health data types	Heart Rate: BLE_KEY HEART_RATE Blood Oxygen: BLEKEY BLOOD OXYGEN HRV: BLE_KEY HRV Stress: BLE_KEY_STRESS Blood Sugar: BLE_KEY_BLOOD_SUGAR Blood Pressure: BLE_KEY_BLOOD_PRESSURE
type	NSInteger	Start/Stop	Start: 1 Stop: 0

Example of usage:

```
+ (void)controlOpen:(NSInteger)type dataType:(NSInteger)dataType block:(void(^)(int code, id
data))block
//启动心率测试
[DHBleCommand controlOpen:1 dataType:BLE_KEY_HEART_RATE block:^(int code, id  _Nonnull data) {

}];

//关闭心率测试
[DHBleCommand controlOpen:0 dataType:BLE_KEY_HEART_RATE block:^(int code, id  _Nonnull data) {

}];

// 监听测量中实时数值改变
[[NSNotificationCenter defaultCenter] addObserver:self
selector:@selector(updateRingMeasureValueChange:)
name:BluetoothNotificationHealthRingMeasureValueChange object:nil];

- (void)updateRingMeasureValueChange:(NSNotification *)ntf
{
    NSDictionary *tUserInfo = ntf.userInfo;
    NSInteger tDataValue = [tUserInfo[@"dataValue"] integerValue];
    NSInteger tDataType = [tUserInfo[@"dataType"] integerValue];


    NSLog(@"updateRingMeasureValueChange 0x%04X value: %zd", (unsigned int)tDataType, tDataValue);
    if (tDataType == BLE_KEY_APP_REAL_TIME_MUSLIM_COUNT){ //Muslim Count

    }
    else if (tDataType == BLE_KEY_APP_REAL_BLOOD_SUGAR_DATA){ //BloodSugar
```

```
    }
    else if (tDataType == BLE_KEY_APP_REAL_TIME_HRV_DATA){ //HRV

    }
    else if (tDataType == BLE_KEY_APP_REAL_TIME_HR_DATA){ //HR Heart Rate

    }
    else if (tDataType == BLE_KEY_APP_REAL_TIME_BLOOD_OXYGEN_DATA){ //BloodOxygen

    }
    else if (tDataType == BLE_KEY_APP_REAL_TIME_STRESS_DATA){ //Stress

    }
    else if (tDataType == BLE_KEY_APP_REAL_TIME_BP_DATA){ //BloodPressure
        NSInteger sp = [tUserInfo[@"systolic"] integerValue]; //Systolic
        NSInteger dp = [tUserInfo[@"diastolic"] integerValue]; //Diastolic
        NSLog(@"BloodPressure sp=%zd dp=%zd", sp, dp);
    }
}
```

3.2.2.2 Continuous monitoring - Set the interval for continuous monitoring of health data.

> Set the monitoring interval for health data (heart rate, blood oxygen, HRV, stress, blood glucose) throughout the day, in minutes.
>
> **Notes: Currently, only the heart rate interval can be set to 30 minutes or 60 minutes; other parameters (blood oxygen, HRV, stress, blood glucose) can only be set to on or off. The start and end times are fixed to cover the entire day and cannot be modified.**

3.2.2.2.1 Heart rate detection settings and retrieval

> The only interval options available for heart rate monitoring are 30 minutes and 60 minutes;

Method Description:

`+(void)setHeartRateMode:(DHHeartRateModeSetModel *)model block:(void(^)(int code, id data))block`

`+(void)getHeartRateMode:(void(^)(int code, id data))block`

Parameter Description:

Parameter	Type		
model	DHHeartRateModeSetModel	class	isOpen: true (on)/false (off) interval: interval time 30 or 60 minutes startHour: 0 (fixed, cannot be modified) startMin: 0 (fixed, cannot be modified) endHour: 23 (fixed, cannot be modified) endMin: 59 (fixed, cannot be modified);

Example of usage:

```
// 1. Set HeartRate Monitor(设置心率监听)
DHHeartRateModeSetModel *tModeSetModel = [[DHHeartRateModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
```

```
tModeSetModel.startHour = 00; //开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 30; //可设置30分钟或60分钟
[DHBleCommand setHeartRateMode:tModeSetModel block:^(int code, id  _Nonnull data) {
  if (code == 0){
    NSLog(@"setHeartRateMode OK");
  }
}];

//1. 获取心率监听
[DHBleCommand getHeartRateMode:^(int code, id  _Nonnull data) {
  if (code == 0){
    DHHeartRateModeSetModel *model = data;
    NSLog(@"getHeartRateMode OK 开关 %d 检测周期 %d", model.isOpen, model.interval);
  }
}];
```

3.2.2.2.2 Blood oxygen monitoring settings and data retrieval

> The interval for blood oxygen measurements can only be set to 60 minutes;

Method Description:

`+(void)setBoMode:(DHBoModeSetModel *)model block:(void(^)(int code, id data))block`

`+(void)getBoMode:(void(^)(int code, id data))block`

Parameter Description:

Parameter	Type		
model	DHBoModeSetModel	Class	isOpen: true (on)/false (off) interval: Fixed interval of 60 minutes startHour: 0 (fixed, cannot be modified) startMin: 0 (fixed, cannot be modified) endHour: 23 (fixed, cannot be modified) endMin: 59 (fixed, cannot be modified);

Example of usage:

```
// 2. Set Blood oxygen Monitor(设置血氧监听)
DHBoModeSetModel *tModeSetModel = [[DHBoModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00; //开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60; //固定不可设置
[DHBleCommand setBoMode:tModeSetModel block:^(int code, id  _Nonnull data) {
  if (code == 0){
    NSLog(@"setBoMode OK");
  }
}];

//2. 获取血氧监听
[DHBleCommand getBoMode:^(int code, id  _Nonnull data) {
```

```
  if (code == 0){
    DHBoModeSetModel *model = data;
    NSLog(@"getBoMode OK 开关 %d 检测周期 %d", model.isOpen, model.interval);
  }
}];
```

3.2.2.2.3 Heart Rate Variability (HRV) Measurement Settings and Data Acquisition

> The HRV interval can only be set to 60 minutes;

Method Description:

`+(void)setHrvMode:(DHHrvModeSetModel *)model block:(void(^)(int code, id data))block`

`+(void)getHrvMode:(void(^)(int code, id data))block`

Parameter Description:

Parameter	Type		
model	DHHrvModeSetModel	Class	isOpen: true (on)/false (off) interval: Fixed interval of 60 minutes startHour: 0 (fixed, cannot be modified) startMin: 0 (fixed, cannot be modified) endHour: 23 (fixed, cannot be modified) endMin: 59 (fixed, cannot be modified);

Example of usage:

```
// 3. Set HRV Monitor(设置HRV监听)
DHHrvModeSetModel *tModeSetModel = [[DHHrvModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00;//开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60; //固定不可设置
[DHBleCommand setHrvMode:tModeSetModel block:^(int code, id  _Nonnull data) {
  if (code == 0){
    NSLog(@"setHrvMode OK");
  }
}];

//3. 获取HRV监听
[DHBleCommand getHrvMode:^(int code, id  _Nonnull data) {
  if (code == 0){
    DHHrvModeSetModel *model = data;
    NSLog(@"getHrvMode OK 开关 %d", model.isOpen);
  }
}];
```

3.2.2.2.4 Stress detection settings and retrieval

> The interval stress can only be set to 60 minutes;

Method Description:

`+(void)setStressMode:(DHStressModeSetModel *)model block:(void(^)(int code, id data))block`

`+(void)getStressMode:(void(^)(int code, id data))block`

Parameter Description:

Parameter	Type		
model	DHStressModeSetModel	Class	isOpen: true (on)/false (off) interval: Fixed interval of 60 minutes startHour: 0 (fixed, cannot be modified) startMin: 0 (fixed, cannot be modified) endHour: 23 (fixed, cannot be modified) endMin: 59 (fixed, cannot be modified);

Example of usage:

```
// 4. 设置压力监听
DHStressModeSetModel *tModeSetModel = [[DHStressModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00;//开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60; //固定不可设置
[DHBleCommand setStressMode:tModeSetModel block:^(int code, id  _Nonnull data) {
  if (code == 0){
    NSLog(@"setStressMode OK");
  }
}];

//4. 获取压力监听
[DHBleCommand getStressMode:^(int code, id  _Nonnull data) {
  if (code == 0){
    DHStressModeSetModel *model = data;
    NSLog(@"getStressMode OK 开关 %d", model.isOpen);
  }
}];
```

3.2.2.2.5 Blood glucose monitoring settings and data retrieval

> The interval between blood glucose measurements can only be set to 60 minutes;

Method Description:

`+(void)setBloodSugarMode:(DHBloodSugarModeSetModel *)model block:(void(^)(int code, id data))block`

`+(void)getBloodSugarMode:(void(^)(int code, id data))block`

Parameter Description:

Parameter	Type		
model	DHBloodSugarModeSetModel	Class	isOpen: true (on)/false (off) interval: Fixed interval of 60 minutes startHour: 0 (fixed, cannot be modified) startMin: 0 (fixed, cannot be modified) endHour: 23 (fixed, cannot be modified) endMin: 59 (fixed, cannot be modified);

Example of usage:

```
// 5. 设置血糖监听
DHBloodSugarModeSetModel *tModeSetModel = [[DHBloodSugarModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00;//开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60; //固定不可设置
[DHBleCommand setBloodSugarMode:tModeSetModel block:^(int code, id  _Nonnull data) {
  if (code == 0){
    NSLog(@"setBloodSugarMode OK");
  }
}];

//5. 获取血糖监听
[DHBleCommand getBloodSugarMode:^(int code, id  _Nonnull data) {
  if (code == 0){
    DHBloodSugarModeSetModel *model = data;
    NSLog(@"getBloodSugarMode OK 开关 %d", model.isOpen);
  }
}];
```

3.2.2.2.6 Blood pressure monitoring settings and data retrieval

> The interval for blood pressure measurements can only be set to 60 minutes;
>
> Configuration table property: `isDataTypeBloodPressure`

Method Description:

`+(void)setBpMode:(DHBpModeSetModel *)model block:(void(^)(int code, id data))block`

`+(void)getBpMode:(void(^)(int code, id data))block`

Parameter Description:

Parameter	Type		
model	DHBpModeSetModel	Class	isOpen: true (on)/false (off) interval: Fixed interval of 60 minutes startHour: 0 (fixed, cannot be modified) startMin: 0 (fixed, cannot be modified) endHour: 23 (fixed, cannot be modified) endMin: 59 (fixed, cannot be modified);

Example of usage:

```
// 6. Set Blood Pressure Monitor(设置血压监听)
DHBpModeSetModel *tModeSetModel = [[DHBpModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00;//开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60; //固定不可设置
[DHBleCommand setBpMode:tModeSetModel block:^(int code, id  _Nonnull data) {
  if (code == 0){
    NSLog(@"setBpMode OK");
  }
}];

//6. Get Blood Pressure Monitor
[DHBleCommand getBpMode:^(int code, id  _Nonnull data) {
  if (code == 0){
    DHBpModeSetModel *model = data;
    NSLog(@"getBpMode OK isOpen %d", model.isOpen);
  }
}];
```

3.2.2.3 24/7 monitoring - Synchronized health history data

> Synchronizing health history data will automatically sync the corresponding health data based on the device's capabilities.
>
> `data` is an array containing multi-day data of a specific type. It will return data of that type sequentially; `data` will not contain data of multiple types.

```
[DHBleCommand startDataSyncing:^(int code, id data){
                NSLog(@"同步完成 %d", code);
            } datablcok:^(int code, int progress, id  _Nonnull data) {
                if (code == 0) {
                    if ([data isKindOfClass:[NSArray class]]) {
                        NSArray *array = data;
                        for (id model in array) {
                            if ([model isKindOfClass:[DHDailyStepModel class]]) {  //Step
                                NSLog(@"同步有 计步数据");
                            }
                            else if ([model isKindOfClass:[DHDailySleepModel class]]) { //Sleep
                                NSLog(@"同步有 睡眠数据");
                            }
```

```
                            else if ([model isKindOfClass:[DHDailyHrModel class]]) { //HeartRate
                                NSLog(@"同步有 心率数据");
                            }
                            else if ([model isKindOfClass:[DHDailyBoModel class]]) { //BO
                                NSLog(@"同步有 血氧数据");
                            }
                            else if ([model isKindOfClass:[DHDailyHrvModel class]]) { ///HRV
                                NSLog(@"同步有 HRV数据");
                            }
                            else if ([model isKindOfClass:[DHDailyPressureModel class]]) { ///Stress
                                NSLog(@"同步有 Stress数据");
                            }
                            else if ([model isKindOfClass:[DHDailyBloodSugarModel class]]) {
///BloodSugar
                                NSLog(@"同步有 血糖数据");
                            }
                            else if ([model isKindOfClass:[DHDailyMuslimCountModel class]]) {
///Muslim count
                                NSLog(@"同步有 赞念数据");
                            }
                        }
                    }
                }
            }];
```

3.2.2.4 24/7 Monitoring - Health Data Explanation

1. Today's and historical step data - DHDailyStepModel

   Both today's and historical step data use `DHDailyStepModel`. Today's data normally contains one model for the current day; historical data may contain multiple dated models, one per day.

Data	Returned content	Daily totals
Today	One `DHDailyStepModel` for the current day	Uses the daily total steps, calories, and distance returned by the device
History	May contain multiple dated `DHDailyStepModel` objects	Sums the historical details for steps, calories, and distance

```
@interface DHDailyStepModel : NSObject

/// 日期时间戳（秒）
@property (nonatomic, copy) NSString *timestamp;
/// 日期yyyyMMdd
@property (nonatomic, copy) NSString *date;

/// 里程（米）
@property (nonatomic, assign) NSInteger distance; //当天的总距离
/// 消耗（卡路里）
@property (nonatomic, assign) NSInteger calorie; //当天总卡路里
/// 步数（步）
@property (nonatomic, assign) NSInteger step; //当天总步数

/// Detail interval in minutes; defaults to 60 for old devices
```

```
@property (nonatomic, assign) NSInteger activityDataInterval;

/// Each item contains timestamp, index, step, calorie and distance
/// timestamp is a Unix timestamp in seconds; index is the detail slot within the day
@property (nonatomic,strong) NSMutableArray <NSDictionary *>*items;

@end
```

`activityDataInterval` is the interval, in minutes, between step details in `items`. A value of `60` means one detail per hour, and `10` means one detail every 10 minutes. The default is `60` when unconfigured. Use `timestamp` as the precise detail time.

2. Sleep data - DHDailySleepModel

```
@interface DHDailySleepModel : NSObject

/// 时间戳（秒）
@property (nonatomic, copy) NSString *timestamp;
/// 日期yyyyMMdd
@property (nonatomic, copy) NSString *date;

/// 总时长（分钟）Total duration (minutes)
@property (nonatomic, assign) NSInteger duration;
/// 入睡时间（时间戳（秒））Time of falling asleep (timestamp in seconds)
@property (nonatomic, copy) NSString *beginTime;
/// 醒来时间（时间戳（秒））Wake-up time (timestamp in seconds)
@property (nonatomic, copy) NSString *endTime;

/// 睡眠项 例：@[@{@"status":@0,@"value":@60},...]
/// status（睡眠类型）value（时长（分钟））
/// status (0.awake 1. light sleep 2. deep sleep 3. REM)
@property (nonatomic,strong) NSMutableArray <NSDictionary *>*items;

@end
```

3. Heart rate data - DHDailyHrModel;

```
@interface DHDailyHrModel : NSObject

/// 时间戳（秒）
@property (nonatomic, copy) NSString *timestamp;
/// 日期yyyyMMdd
@property (nonatomic, copy) NSString *date;

/// 心率项 例：@[@{@"timestamp":@0,@"value":@80},...]
/// timestamp（时间戳（秒））value（心率值）
@property (nonatomic,strong) NSMutableArray <NSDictionary *>*items;


@end
```

HRV`DHDailyHrvModel`，压力`DHDailyPressureModel`，血氧`DHDailyBoModel`,血糖`DHDailyBloodSugarModel`与心率类似可参考对应类

4. Muslim prayer data - DHDailyMuslimCountModel

```
@interface DHDailyMuslimCountModel : NSObject
/// 时间戳（秒）
@property (nonatomic, copy) NSString *timestamp;
/// 日期yyyyMMdd
@property (nonatomic, copy) NSString *date;

@property (nonatomic, assign) NSInteger muslimcount;

/// 赞念项 例：@[@{@"timestamp":@0,@"value":@80},...]
/// timestamp（时间戳（秒））value（每小时赞念值）
@property (nonatomic,strong) NSMutableArray <NSDictionary *>*items; //注意,这里的每小时为累加值赞念

@end
```

3.2.3 OTA upgrade

> **Note**
>
> The OTA update file must be obtained from the manufacturer and tested thoroughly before proceeding. This is to prevent update errors and device malfunction.

Method Description:

`+(void)ringOtaWithFileData:(NSData *)fileData block:(void(^)(int code, CGFloat progress, id data))block`

Parameter Description:

Parameter	Type	Description
fileData	NSData	Firmware file data
block	Block	Progress and result callback, code=0 in progress, progress is 0-1

Example of usage:

```
NSString *tFilePath = @""; //bin file path, provided by manufacturer
NSData *fileData = [NSData dataWithContentsOfFile:tFilePath];
[DHBleCommand ringOtaWithFileData:fileData block:^(int code, CGFloat progress, id  _Nonnull data) {
    NSLog(@"OTA code %d progress %.2f", code, progress);
}];
```

3.2.4 Exercise more

> **Caution**
>
> The multi-sport configuration table property is `isSupportWorkout3`. After enabling multi-sport mode, the device will enter exercise mode. Neither disconnecting the app nor closing it will stop the activity; it can only be stopped manually through the app or the device itself. Therefore, for devices with multi-sport functionality, please check the status after connecting to determine if it is currently in exercise mode, as this may affect the use of other functions.
>
> **The exercise duration must exceed 2 minutes for the device to save the workout data.**

3.2.4.1 Get the device's multiple motion states.

> Check if the device is currently engaged in multiple activities; only start a new activity if it's not currently engaged in any activity.

Method Description:

`+(void)getControlSportWithRing:(void(^)(int code, id data))block`

Parameter Description:

WorkoutControlType	Type		
Workout_Begin	Int	整形	Begin
Workout_Continue	Int	整形	Continue
Workout_Pause	Int	整形	Pause
Workout_Finish	Int	整形	Finish

Example of usage:

```
[DHBleCommand getControlSportWithRing:^(int code, id  _Nonnull data) {
  // @{@"keySportType":@(tSportType), @"keyControlType":@(tControlType)}
  if (code == 0 && [data isKindOfClass:[NSDictionary class]]){
    NSDictionary *tDic = data;
    NSInteger tSportType = [tDic[@"keySportType"] integerValue];
    WorkoutControlType tControlType = [tDic[@"keyControlType"] integerValue];
  }];
```

3.2.4.2 The control device enters multi-motion mode.

> The control device enters multi-motion mode and starts the motion.
>
> Changes in exercise data are obtained by receiving the `BluetoothNotificationRingRuningData` notification.

Method Description:

`+(void)controlSportWithRing:(DHSportControlModel *)model block:(void(^)(int code, id data))block`

Parameter Description:

Parameter	Type		
DHSportControlModel	Class		controlType: refer to WorkoutControlType sportType: refer to BleActivityMode

Explanation of the data returned in the notification of changes in exercise data:

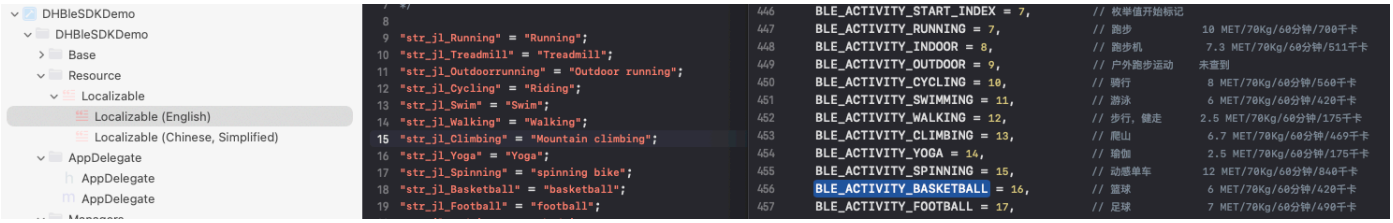
Parameter	Type		
ActivityTime	Int	整形	Duration of exercise, in seconds (s);
ActivitySteps	Int	整形	Steps taken during exercise
ActivityDistance	Int	整形	Distance is generated during movement, measured in meters (m);
ActivityCalorie	Int	整形	Heat is generated during exercise, measured in calories (cal);
ActivityHr	Int	整形	Dynamic heart rate during exercise
ActivityDataType	Int	整形	The source type, which is also returned by `setRingEnterWorkOut`.

Example of usage:

```
DHSportControlModel *model = [[DHSportControlModel alloc] init];
model.controlType = Workout_Begin; //开始
model.sportType = tbleActivityMode;
[DHBleCommand controlSportWithRing:model block:^(int code, id  _Nonnull data) {
  if (code == 0){
    WorkoutRunningController *runningC = [[WorkoutRunningController alloc]
initWithNibName:@"WorkoutRunningController" bundle:nil];
    runningC.bleActivityMode = tbleActivityMode;
    runningC.controllType = Workout_Begin; //开始
    runningC.modalPresentationStyle = UIModalPresentationOverFullScreen;
    [weakSelf presentViewController:runningC animated:YES completion:^{

    }];
  }
}];


[[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(ringRuningDataUdpate:)
name:BluetoothNotificationRingRuningData object:nil];

- (void)ringRuningDataUdpate:(NSNotification *)ntf
{
    NSDictionary *tUserInfo = ntf.userInfo;
    if (tUserInfo.count > 0){
        NSInteger duration = [tUserInfo[@"ActivityTime"] integerValue];
        NSInteger totalStep = [tUserInfo[@"ActivitySteps"] integerValue];
        NSInteger activityDistance = [tUserInfo[@"ActivityDistance"] integerValue];
        NSInteger activityCalorie = [tUserInfo[@"ActivityCalorie"] integerValue];
        NSInteger activityHr = [tUserInfo[@"ActivityHr"] integerValue];
        NSInteger tActivityDataType = [tUserInfo[@"ActivityDataType"] integerValue];

        NSLog(@"ringRuningDataUdpate tActivityDataType %04X", (UInt32)tActivityDataType);
        NSInteger hours = duration / 3600;
        NSInteger minutes = (duration % 3600) / 60;
        NSInteger seconds = duration % 60;
        self.workoutTimeLb.text = [NSString stringWithFormat:@"Time: %02ld:%02ld:%02ld", (long)hours,
(long)minutes, (long)seconds];
        self.workoutStepLb.text = [NSString stringWithFormat:@"Steps: %ld", totalStep];
        self.workoutDistanceLb.text = [NSString stringWithFormat:@"Distance: %.2f Km",
floor(activityDistance/1000.0*100)/100];
```

```
        self.workoutCaloriesLb.text = [NSString stringWithFormat:@"Calories: %.1f KCal",
floor(activityCalorie/1000.0*10)/10];
        self.workoutHeartLb.text = [NSString stringWithFormat:@"HeartRate: %ld bpm", activityHr];
    }
}
```

The corresponding names for BleActivityMode can be found in the example Demo string definitions:



3.2.4.3 Control enabling/disabling real-time notifications of motion data from the device.

> 控制开启/关闭设备实时通知运动数据;
>
> 运动中数据变化通过接收 `BluetoothNotificationRingRuningData` 通知获取,有时app关闭与进入后台,可告诉设备停止通知数据.

Method Description:

`+(void)setRingEnterWorkOut:(UInt8)isEnter block:(void(^)(int code, id data))block`

Parameter Description:

Parameter	Type		
isEnter	Int		1: Enable notification of exercise data; 0: Disable notification of exercise data;

Example of usage:

```
//退出运动界面
[DHBleCommand setRingEnterWorkOut:0 block:^(int code, id  _Nonnull data) {

}];
```

3.2.4.4 Obtain multi-sport data reports.

Method Description:

`+(void)startRingWorkout3Syncing:(void(^)(int code, id data))block dataBlock:(void(^)(int code, int progress, id data))dataBlock`

Return data DHDailySportModel parameter description:

DHDailySportModel 类	Type		
timestamp	NSString		Exercise start timestamp
date	NSString		Date-yyyyMMdd
viewType	Int		Current exercise types include: steps, cadence, pace, and distance: With cadence: viewTypeHaveStepFaq: Without step count: viewTypeNoStepNum: With pace: viewTypeHavePace: Without distance: viewTypeNoDistance:
heartRateItems	Array		The current list of heart rates generated during exercise, saved every minute;
pacePerKmItems	Array		Pace per kilometer list, unit: seconds/km; empty if device does not support
.....			Other attributes can be found in the class documentation.

Example of usage:

```
[DHBleCommand startRingWorkout3Syncing:^(int code, id  _Nonnull data) {
  NSLog(@"startRingWorkout3Syncing 同步完成 code %d", code);
} dataBlock:^(int code, int progress, id  _Nonnull data) {
  if (code == 0) {
    if ([data isKindOfClass:[NSArray class]]){
      NSArray *array = data;
      for (id model in array) {
        if ([model isKindOfClass:[DHDailySportModel class]]) {
          //保存数据库或其它操作

        }
      }

      NSLog(@"startRingWorkout3Syncing data %zd", array.count);

    }
  }
}];
```

5.2.5 Sensor Raw Data

> PPG/ACC/PPG Red/IR sensor raw data collection and sleep real-time data;
>
> Configuration table properties: `isSupportSensorRawPPG` (PPG), `isSupportSensorRawACC` (ACC), `isSupportSensorRawPPGRed` (PPG Red), `isSupportSensorRawIR` (IR), `isSupportSensorRawSleep` (Sleep real-time);
>
> **Note: Sleep real-time data (sensorType=5) does not require manual start/stop. When the device supports this feature, it will automatically push data during sleep. Receive it via the same notification `BluetoothNotificationHealthRingSenorRawChange`.**

sensorType valid combinations:

Value	Meaning	Description
1	ACC	ACC only
2	PPG Green	PPG Green only
3	PPG Green + ACC	PPG Green and ACC simultaneously
4	PPG Red	PPG Red only
5	PPG Red + ACC	PPG Red and ACC simultaneously
10	PPG Green + IR	PPG Green and IR simultaneously
11	PPG Green + ACC + IR	PPG Green, ACC and IR simultaneously
12	PPG Red + IR	PPG Red and IR simultaneously
13	PPG Red + ACC + IR	PPG Red, ACC and IR simultaneously

> **Rules: PPG Green and PPG Red cannot coexist; IR cannot start alone, must be combined with PPG Green or PPG Red.**

Return Data format:

Field	Description
sensorType	Type: 1=PPG, 2=ACC, 3=PPG Red, 4=IR, 5=Sleep real-time
ppgData	PPG data array, each item is int32
accData	ACC data array, each item is {x,y,z} (int16)
ppgRedData	PPG Red data array, each item is int32
irData	IR infrared data array, each item is int32
sleepData	Sleep data array when type=5, each item is {timestamp, mode}; mode: 17=Start, 34=End, 1=Deep, 2=Light, 3=Awake, 4=REM

5.2.5.0 PPG Timed Monitoring

> PPG timed monitoring setting, similar to heart rate/HRV timed monitoring;
>
> Configuration table property: `isSupportPPGMonitoring`

Method Description:

```
+(void)setPPGMode:(DHHrvModeSetModel *)model block:(void(^)(int code, id data))block
```

```
+(void)getPPGMode:(void(^)(int code, id data))block
```

Parameter Description:

Parameter	Type	Description	Value
model	DHHrvModeSetModel	Class	isOpen: true on/false off interval: default 30 minutes startHour: 0 fixed startMinute: 0 fixed endHour: 23 fixed endMinute: 59 fixed

Example of usage:

```
//Set PPG monitoring
DHHrvModeSetModel *tModeSetModel = [[DHHrvModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00;
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23;
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60;
[DHBleCommand setPPGMode:tModeSetModel block:^(int code, id  _Nonnull data) {
    if (code == 0){ NSLog(@"setPPGMode OK"); }
}];

//Get PPG monitoring
[DHBleCommand getPPGMode:^(int code, id  _Nonnull data) {
    if (code == 0){
        DHHrvModeSetModel *model = data;
        NSLog(@"getPPGMode OK isOpen %d", model.isOpen);
    }
}];
```

5.2.5.1 Start and Stop Sensor Raw Data

> block callback with code==0 indicates start/stop success;
>
> The device may also stop the sensor actively, notified via `BluetoothNotificationHealthRingSenorStopChange`.

Method Description:

`+(void)ringControlSensorRaw:(UInt8)outputType type:(UInt8)sensorType block:(void(^)(int code, id data))block`

Parameter Description:

Parameter	Type	Description	Value
outputType	UInt8	Output control type	1: Start Sensor output 2: Stop Sensor output
sensorType	UInt8	Sensor type (bitmask)	See valid combinations table above

Example of usage:

```
//Listen for device-initiated sensor stop
[[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(sensorStopChange:)
name:BluetoothNotificationHealthRingSenorStopChange object:nil];

- (void)sensorStopChange:(NSNotification *)ntf {
    NSLog(@"Device stopped sensor actively");
}

//Start PPG+ACC raw data output (sensorType=3)
[DHBleCommand ringControlSensorRaw:1 type:3 block:^(int code, id data) {}];

//Stop PPG+ACC raw data output
[DHBleCommand ringControlSensorRaw:2 type:3 block:^(int code, id data) {}];
```

5.2.5.2 Data Retrieval Methods

> There are two ways to retrieve sensor raw data. **The device determines which method is used, the APP cannot choose**:
>
> (1) Real-time push: After starting, the device pushes data to the APP in real-time;
>
> (2) History retrieval: The device collects and saves data first, then the APP actively syncs it later;

5.2.5.2.1 Real-time Push

> After starting the sensor, the device pushes raw data in real-time;
>
> Data is returned via the `BluetoothNotificationSensorRawData` notification.

Example of usage:

```
[[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(sensorRawDataUpdate:)
name:BluetoothNotificationSensorRawData object:nil];

- (void)sensorRawDataUpdate:(NSNotification *)ntf
{
    NSDictionary *tUserInfo = ntf.userInfo;
    NSInteger tType = [tUserInfo[@"sensorType"] integerValue];
    if (tType == 2) {
        NSLog(@"ACC count=%zd", [tUserInfo[@"accData"] count]);
    } else if (tType == 1) {
        NSLog(@"PPG count=%zd", [tUserInfo[@"ppgData"] count]);
    } else if (tType == 3) {
        NSLog(@"PPG Red count=%zd", [tUserInfo[@"ppgRedData"] count]);
    } else if (tType == 4) {
        NSLog(@"IR count=%zd", [tUserInfo[@"irData"] count]);
    } else if (tType == 5) {
        NSArray *sleepData = tUserInfo[@"sleepData"];
        NSLog(@"Sleep count=%zd", sleepData.count);
    }
}
```

5.2.5.2.2 History Retrieval

> Retrieve historical sensor raw data saved on the device, similar to the multi-sport data sync pattern;
>
> Data is returned via `dataBlock` callback. `block` with `code==0` indicates sync is complete.

Method Description:

`+(void)ringGetHistorySensorRaw:(void(^)(int code, id data))block dataBlock:(void(^)(int code, int progress, id data))dataBlock`

dataBlock returns NSArray, each element contains:

Field	Description
sensorType	Sensor type (1:PPG 2:ACC 3:PPG Red 4:IR)
sequence	Sequence number
count	Data count
ppgData	PPG data array (when sensorType==1)
accData	ACC data array (when sensorType==2), each item has x,y,z
ppgRedData	PPG Red data array (when sensorType==3)
irData	IR data array (when sensorType==4)

> dataBlock is called only once with the complete result array. progress is always 100.

Example of usage:

```
[DHBleCommand ringGetHistorySensorRaw:^(int code, id  _Nonnull data) {
    NSLog(@"Sensor History sync finished code %d", code);
} dataBlock:^(int code, int progress, id  _Nonnull data) {
    if (code == 0 && data) {
        NSArray *resultArray = data;
        for (NSDictionary *info in resultArray) {
            NSLog(@"sensorType=%@ seq=%@ count=%@", info[@"sensorType"], info[@"sequence"],
info[@"count"]);
        }
    }
}];
```

SDK Revision History

V2.0.0_20260724 (2026.07.24)

- Added support for step-detail intervals.

V2.0.0_20260706 (2026.07.06)

- Added external `CBCentralManager` integration (3.1.8), allowing the customer App to scan devices with its unified scanner and let the SDK connect with the same Central.

联系方式 / 技术支持

- 技术支持邮箱 developer@dhouse88.com